



ASSIGNMENT #3

Course: **Compiler Construction**

Instructor: **Ms. Nirmal Tariq**

BY:

Ruhab Ahmad (23I-0559)

Hasan Altaf (23I-0698)

Degree: **Computer Science**

Section: **E**

FAST - NUCES

Introduction

This project implements two core bottom-up parsing algorithms: SLR (1) and LR (1). Bottom-up parsers attempt to construct a parse tree for an input string beginning at the leaves (terminals) and working up towards the root (start symbol) through a series of shift and reduce operations. The assignment explores the differing power of these two approaches, particularly how the inclusion of strict lookahead symbols in LR (1) items allows it to resolve conflicts that SLR (1) cannot handle, as SLR (1) broadly relies on FOLLOW sets.

Approach Data Structures Used: Items: A generic Item struct tracks the production index, the dot position (representing the parsing state), and an optional lookahead string utilized exclusively by the LR (1) parser. Item Sets (States): Implemented using standard set containers to ensure uniqueness of items. Parsing Tables: The ACTION and GOTO tables are mapped using key-value pairs enabling efficient lookup time for state-symbol transitions.

Algorithm Implementation Details

CLOSURE: For LR (0), closure adds new productions whenever a dot precedes a non-terminal. For LR (1), the closure specifically calculates the FIRST set of the string following the non-terminal appended with the item's lookahead.

GOTO: Transitions the dot past a specific grammar symbol and computes the closure of the resulting item. Grammar Augmentation: A new root S-prime is injected pointing to the original start symbol to provide a singular, unambiguous accepting state.

Lookaheads: Lookaheads in LR (1) were handled by storing them directly within the item state and strictly checking them before performing reduce actions.

Challenges

FIRST and FOLLOW Computations: Developing a robust algorithm for FIRST sets that properly handles epsilon productions and recursive loops required careful state tracking. State Equivalency: In LR (1), two items are only identical if their core and their lookahead match. Initial bugs where states merged incorrectly were resolved by updating the equality and comparison logic. Memory Management: Because trees and dynamic collections were heavily utilized, passing data by reference was vital to prevent memory bloat and ensure stable execution.

Test Cases

The system was validated against multiple grammars: Grammar 1 (Simple Expression): Successfully processed valid and invalid strings. Grammar 2 (Multiplication): Successfully generated trees enforcing precedence. Grammar 3 (Classic Conflict): As expected, SLR (1) generated a Shift/Reduce conflict when parsing the grammar. The LR (1) parser successfully resolved this due to stricter lookahead constraints, demonstrating an SLR (1) conflict successfully resolved by LR (1). (Note: 5 inputs were tested per grammar, including valid strings, syntax errors, and empty inputs).

Comparison Analysis SLR (1) vs LR (1) Parsing Power

LR (1) is significantly more powerful as it uses exact lookaheads rather than broad FOLLOW sets, avoiding many Shift/Reduce conflicts.

Number of States: SLR (1) generated far fewer states. LR (1) generated a significantly higher number of states due to the lookahead explosion.

Table Construction Time: SLR (1) tables were constructed quickly. LR (1) took longer due to the larger state space and complex CLOSURE operations.

Memory Usage: SLR (1) had a minimal memory footprint, whereas LR (1) required higher memory usage to store the expanded items and larger parsing tables.

Sample Outputs

Please paste your screenshots of program execution, item sets and parsing tables, parse trees for accepted strings, and conflict resolutions here before converting to PDF.

Conclusion

Implementing these parsers from scratch highlights the direct trade-off between compiler performance and memory overhead. While LR (1) can parse a significantly larger class of context-free grammars by avoiding conflicts, the exponential growth in state sizes makes it computationally expensive to generate compared to SLR (1).